
Workgroup:	Network Working Group
Internet-Draft:	draft-wullink-rpp-extension-guidelines-00
Published:	14 November 2026
Intended Status:	Informational
Expires:	18 May 2027
Authors:	M. Wullink P. Kowalik <i>SIDN Labs DENIC</i>

Extension Guidelines for RESTful Provisioning Protocol (RPP)

Abstract

Extension guidelines for the RESTful Provisioning Protocol (RPP) ...

TODO

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
2. Terminology	3
3. Conventions Used in This Document	3
4. Restrictions on Extensions	3
5. RPP Extension Points	4
5.1. New Data Objects	4
5.2. New Data Elements on Existing Objects	4
5.3. New Associations	4
5.4. New Operations	4
5.5. URL Endpoints Are Derived, Not a Separate Extension Point	4
5.6. Extended Operation Inputs and Outputs	5
5.7. External Data Types	5
5.8. New RPP Result Codes	5
5.9. Error Object Extension Fields	5
5.10. Additional Query Parameters	5
5.11. New Authentication and Authorization Methods	5
5.12. Discovery and Profile Advertisement	5
6. Extending Data Objects Defined in Other Specifications	6
6.1. JSON Schema Composition	6
6.2. Schema Identification	6
6.3. Client Implementation	6
6.3.1. Java	7
6.4. Registration	7
6.5. Validation	7
7. IANA Considerations	7
8. Internationalization Considerations	7
9. Privacy Considerations	7
10. Change History	8

11. Normative References	8
Appendix A. Java Example	8
Acknowledgements	13
Authors' Addresses	13

1. Introduction

TODO

2. Terminology

In this document the following terminology is used.

URL - A Uniform Resource Locator as defined in [\[RFC3986\]](#).

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

Base specification - The RPP specification, such as [\[I-D.ietf-rpp-data-objects\]](#), [\[I-D.ietf-rpp-json\]](#) or [\[I-D.ietf-rpp-core\]](#), that originally defines a data object, component object, or operation.

Extension specification - A specification that adds new data elements, associations, or operations to an object or operation defined in a base specification, without modifying the base specification itself.

Combined schema - The JSON Schema produced by composing the JSON Schema definition of a base object with the JSON Schema contributed by every extension specification supported by a given deployment or profile, as defined in [\[I-D.ietf-rpp-core\]](#).

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [\[RFC2119\]](#).

In examples, indentation and white space are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

4. Restrictions on Extensions

An extension MUST NOT remove, redefine, or otherwise alter the semantics of a property defined by the RPP base specification. An extension MUST only add new properties or associations, and MUST ensure that any new elements do not conflict with existing ones in the base specification.

5. RPP Extension Points

An extension specification MAY extend RPP along one or more of the following independent extension points. Each extension point has its own registration requirements and, where applicable, its own schema-composition method.

5.1. New Data Objects

Extensions MAY define entirely new resource, component, or process objects, as described in [I-D.ietf-rpp-data-objects]. New objects MUST be registered in the RPP Data Object Registry [I-D.ietf-rpp-data-objects] and MUST define their own JSON Schema `$defs` entry with its own `$id`. Because no base object definition exists to compose with, the Additive Schema Composition method described below does not apply to this extension point.

5.2. New Data Elements on Existing Objects

Extensions MAY add new data elements (properties) to a data object, component object, or process object defined in a base specification. This is the extension point covered in detail in the "Extending Data Objects Defined in Other Specifications" section below: the extension MUST compose its new properties onto the base object's JSON Schema definition using `allOf`, and MUST register the addition in the Object and Operation Extension registry defined in [I-D.ietf-rpp-data-objects].

5.3. New Associations

Extensions MAY introduce new associations (Aggregation, Composition, or their Labelled/Dictionary variants) between existing or new objects, as described in [I-D.ietf-rpp-data-objects]. An association is represented as a data element whose Data Type references another object; it MUST follow the same additive schema composition and registration rules as any other new data element.

5.4. New Operations

Extensions MAY define entirely new operations on existing or new data objects. New operations MUST define their own request and response JSON Schema `$defs`, following [I-D.ietf-rpp-json], and MUST be registered in the Object and Operation Extension registry defined in [I-D.ietf-rpp-data-objects].

5.5. URL Endpoints Are Derived, Not a Separate Extension Point

RPP endpoint URLs are not an independent extension point. As defined in [I-D.ietf-rpp-core], every endpoint URL and HTTP method is derived mechanically from a Data Object's "Identifier", from its operations, and from the Direct Access flag on its data elements; no endpoint is defined independently of a corresponding Data Object. Consequently:

- A new Data Object automatically obtains a collection endpoint once its "Identifier" is registered, by applying the `plural()` derivation rule.
- A new association with Direct Access set to `true` automatically obtains a sub-resource path, derived recursively from the parent resource's path.
- A new operation automatically obtains its HTTP method and URL path once its "Identifier" is registered.

An extension specification **MUST NOT** define ad-hoc endpoint URLs or HTTP methods. It **MUST** instead register the underlying data object, association, or operation as described in the corresponding extension point above, and rely on the derivation rules in [\[I-D.ietf-rpp-core\]](#) to determine the resulting endpoint.

5.6. Extended Operation Inputs and Outputs

Extensions **MAY** add transient data elements to the input or output of an existing operation (e.g. an additional field in a Create request or Read response) without adding a persistent property to the underlying data object. Such additions **MUST** follow the same Additive Schema Composition method described below, applied to the operation's request or response \$defs entry, as defined in [\[I-D.ietf-rpp-json\]](#), rather than to the object's own canonical schema.

5.7. External Data Types

Instead of adding properties to an object, an extension **MAY** reference a type defined entirely in an external specification (e.g. `External:RPP-JSContact-Profile:Card`), as described in [\[I-D.ietf-rpp-data-objects\]](#). This is the appropriate mechanism when substituting or supplying a whole, independently versioned data type, rather than adding fields to an existing RPP-defined object.

5.8. New RPP Result Codes

Extensions **MAY** define new RPP result codes within the reserved result code classes, registered in the RPP Result Codes registry defined in [\[I-D.ietf-rpp-core\]](#).

5.9. Error Object Extension Fields

Extensions **MAY** add extension fields to the Problem Detail error object to convey additional, extension-specific information about the cause of an error, as described in [\[I-D.ietf-rpp-core\]](#). These fields **MUST** follow the same additive JSON Schema composition rules as any other new data element.

5.10. Additional Query Parameters

Extensions **MAY** define additional HTTP query parameters for existing operations, for example to further qualify a check or read request, as described in [\[I-D.ietf-rpp-core\]](#).

5.11. New Authentication and Authorization Methods

Extensions **MAY** define additional authentication or authorization methods for use with RPP. Such methods **MUST** be registered in the RPP Extension registry defined in [\[I-D.ietf-rpp-core\]](#).

5.12. Discovery and Profile Advertisement

Regardless of which of the above extension points it uses, a server **MUST** advertise support for an extension in the RPP Discovery document's extensions array (name, id, version, url), as defined in [\[I-D.ietf-rpp-core\]](#), and the extension **MAY** be included in one or more RPP Profiles that bundle a set of supported extensions for a given deployment.

6. Extending Data Objects Defined in Other Specifications

RPP is designed so that extension specifications can add new data elements to data objects, component objects, and operations without modifying the specification that originally defines them, as described in [I-D.ietf-rpp-data-objects]. This section defines the normative method an extension specification MUST follow when it adds properties to an object defined in a base specification, so that JSON Schema validation, as defined in [I-D.ietf-rpp-json], keeps functioning correctly across specifications.

6.1. JSON Schema Composition

For every data object which is modified by an extension specification, the extension specification MUST define a new JSON Schema document that composes the base object's schema with the new properties using `allOf`. The extension specification MUST NOT copy, redefine, or otherwise restate the JSON Schema definition of the base object. Instead, it MUST reference the base object's definition by its fully qualified `$id`-relative pointer (e.g. `https://www.iana.org/.../domainName.json#/$defs/domainName`) in an `allOf` branch, and add its own new properties in a separate branch.

The extension specification MUST include a JSON schema for every operation that is modified by the extension.

6.2. Schema Identification

A specification that defines JSON Schema for RPP objects MUST assign a stable, dereferenceable `$id` to its schema document(s), and MUST NOT change the `$id` of a published `$defs` entry. Extension specifications MUST reference a base object's definition by its fully qualified `$id`-relative pointer (e.g. `https://www.iana.org/.../domainName.json#/$defs/domainName`) rather than copying the base definition into their own schema.

6.3. Client Implementation

A client does not need to generate a distinct set of code classes for every base object and extension combination offered by a particular server. Because extension schemas are additive, self-contained, and independent of one another, a client implementation SHOULD instead maintain one set of code classes for each base object and a separate set of code classes for each extension it supports, keeping an extension's classes distinct from the base object's classes rather than flattening their properties together. Composing a base object with whichever extensions apply then becomes a runtime decision, rather than something fixed by code generation for every individual server.

A client learns at runtime which extensions a given server supports from the `extensions` list in that server's RPP Discovery response, as defined in [I-D.ietf-rpp-core]. It SHOULD attach, populate, and validate extension class instances only for the extensions the server has advertised as supported, and SHOULD ignore any properties in a response that belong to an extension it does not recognize or does not support.

Because an extension that is not attached to a given base object instance contributes no properties at all, a client naturally serializes only the base object together with the properties of whichever extensions are actually in use, without needing to distinguish this case from a property that is merely absent or `null` within a supported extension. Attached extensions are merged flat with the base object on the wire, consistent with the schema

composition described in "JSON Schema Composition". Within an attached extension, an optional property without a value continues to follow Rule 2 and is omitted from the output rather than represented as `null`.

6.3.1. Java

For Java clients, common JSON serialization frameworks such as Jackson support this attach-then-merge model directly: extension instances can be merged into the base object's serialized tree at runtime (e.g. `ObjectNode.setAll()`), inlined via a nullable field annotated `@JsonUnwrapped`, or flattened through `@JsonAnyGetter/@JsonAnySetter`-backed property maps. In all three cases, an extension that is not attached contributes no properties, and `@JsonInclude(Include.NON_NULL)` on each extension class independently omits its own unset optional properties. A property that an extension's schema defines as required can be annotated `@JsonProperty(required = true)`, causing Jackson to reject an incoming JSON instance during deserialization if the property is missing; because this check only applies when the extension class itself is being deserialized, it naturally does not apply to servers or requests where that extension is not in use. Required-property validation for an extension is otherwise typically out of scope for the JSON library itself and is instead enforced with a Bean Validation framework (e.g. `jakarta.validation`) or a JSON Schema validator, applied only to attached extension instances.

6.4. Registration

An extension specification that adds properties to an existing object **MUST** register the added data elements in the Object and Operation Extension registry defined in [\[I-D.ietf-rpp-data-objects\]](#). The registration **MUST** additionally include a dereferenceable URL to the JSON Schema document defining the `$defs` entry described in the Additive Schema Composition section above, and **MUST** identify the base object type(s) being extended by their registered identifier.

6.5. Validation

A deployment that supports a specific combination of extensions (a profile, as defined in [\[I-D.ietf-rpp-core\]](#)) **MUST** build, for every extended object type, a combined schema consisting of the base object's definition and the `allOf` branch contributed by every supported extension registered for that object type.

Before using a combined schema to validate a JSON instance, implementations **MUST** inject `"unevaluatedProperties": false` at every object-schema node of the combined schema, following the algorithm described in [\[I-D.ietf-rpp-json\]](#). This injection **MUST** be performed only on the combined, per-deployment schema, and **MUST NOT** be present in the schema document published by any individual base or extension specification. This ensures undeclared properties are rejected while any supported combination of registered extensions validates successfully.

7. IANA Considerations

TODO

8. Internationalization Considerations

TODO

9. Privacy Considerations

TODO

10. Change History

TODO

11. Normative References

- [I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>>.
- [I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>>.
- [I-D.ietf-rpp-json] Wullink, M. and P. Kowalik, "JSON for RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-json-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-json-00>>.
- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <<https://www.rfc-editor.org/info/rfc3986>>.

Appendix A. Java Example

This appendix is informative and illustrates one possible way to implement the attach-then-merge model described in "Client Implementation" using Java and the Jackson library. The base object and each extension are modeled as separate classes; an extension contributes properties to the serialized output only when an instance of it has been attached, and only for extensions the target server has advertised support for. The code below compiles against `jackson-databind` and `jackson-annotations`.

```
import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;
import com.fasterxml.jackson.annotation.JsonUnwrapped;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;

import java.util.List;

// Base object class, one per RPP object type.
public class DomainName {

    @JsonProperty("@type")
    private final String type = "domainName";

    private String name;

    public String getName() {
        return name;
    }
}
```

```
    }

    public void setName(String name) {
        this.name = name;
    }
}

// Extension class, independent of the base object and of other extensions.
@JsonInclude(JsonInclude.Include.NON_NULL)
public class LaunchExtension {

    @JsonProperty(required = true)
    private String phase;

    private String applicationId;

    public String getPhase() {
        return phase;
    }

    public void setPhase(String phase) {
        this.phase = phase;
    }

    public String getApplicationId() {
        return applicationId;
    }

    public void setApplicationId(String applicationId) {
        this.applicationId = applicationId;
    }
}

// Serializes the base object merged with only the attached extension
instances.
public final class RppSerializer {

    private RppSerializer() {
    }

    public static JsonNode serialize(ObjectMapper mapper, Object base,
    List<Object> attachedExtensions) {
        ObjectNode node = (ObjectNode) mapper.valueToTree(base);
        for (Object extension : attachedExtensions) {
            node.setAll((ObjectNode) mapper.valueToTree(extension));
        }
        return node;
    }
}
```

Example usage, attaching an extension only when the target server has advertised support for it:

```
ObjectMapper mapper = new ObjectMapper();
mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);

DomainName domainName = new DomainName();
domainName.setName("example.example");

List<Object> attached = new ArrayList<>();
if (serverSupportedExtensions.contains("launch")) {
    LaunchExtension launch = new LaunchExtension();
    launch.setPhase("sunrise");
    attached.add(launch);
}

JsonNode json = RppSerializer.serialize(mapper, domainName, attached);
```

Because `LaunchExtension` is only added to `attached` when the server supports it, its properties never appear in the output for a server that does not; `JsonInclude.Include.NON_NULL` independently ensures that any of its own unset optional properties are omitted rather than serialized as `null`, consistent with Rule 2 in [\[I-D.ietf-rpp-json\]](#). The `@JsonProperty(required = true)` annotation on `phase` causes Jackson to reject an incoming `LaunchExtension` JSON instance during deserialization if `phase` is missing; this check only runs when a `LaunchExtension` instance is being deserialized, so it has no effect on servers or requests that do not use the extension.

Keeping the extension in its own Java class does not imply that it is serialized as a nested JSON object. When the set of extensions is known at compile time, `@JsonUnwrapped` achieves the same flat, on-the-wire composition without manual tree merging: the `LaunchExtension` instance remains a distinct, independently validated class, but its properties are inlined directly into the enclosing object at the same level as `name`, exactly as required by the JSON Schema `allOf` composition:

```
import com.fasterxml.jackson.annotation.JsonProperty;
import com.fasterxml.jackson.annotation.JsonUnwrapped;

public class DomainNameWithLaunch {

    @JsonProperty("@type")
    private final String type = "domainName";

    private String name;

    // Null when the target server does not support, or the client does
    // not use, the "launch" extension; its properties are then omitted
    // entirely rather than nested under a "launch" object.
    @JsonUnwrapped
    private LaunchExtension launch;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public LaunchExtension getLaunch() {
        return launch;
    }

    public void setLaunch(LaunchExtension launch) {
        this.launch = launch;
    }
}
```

The following JUnit 5 test verifies that an unattached extension contributes no properties, that an attached extension is merged flatly and omits its own unset properties, and that the @JsonUnwrapped form produces the same flat composition:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.exc.MismatchedInputException;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import org.junit.jupiter.api.Test;

class RppSerializerTest {
```

```
private final ObjectMapper mapper = new ObjectMapper()
    .setSerializationInclusion(JsonInclude.Include.NON_NULL);

@Test
void unattachedExtensionContributesNoProperties() {
    DomainName domainName = new DomainName();
    domainName.setName("example.example");

    JsonNode json = RppSerializer.serialize(mapper, domainName,
Collections.emptyList());

    assertEquals("example.example", json.get("name").asText());
    assertFalse(json.has("phase"));
    assertFalse(json.has("applicationId"));
}

@Test
void attachedExtensionIsMergedFlatAndOmitsUnsetProperty() {
    DomainName domainName = new DomainName();
    domainName.setName("example.example");

    LaunchExtension launch = new LaunchExtension();
    launch.setPhase("sunrise");
    // applicationId intentionally left unset (null) and MUST be omitted.

    List<Object> attached = new ArrayList<>();
    attached.add(launch);

    JsonNode json = RppSerializer.serialize(mapper, domainName,
attached);

    assertEquals("example.example", json.get("name").asText());
    assertEquals("sunrise", json.get("phase").asText());
    assertFalse(json.has("applicationId"));
}

@Test
void jsonUnwrappedProducesTheSameFlatComposition() {
    DomainNameWithLaunch domainName = new DomainNameWithLaunch();
    domainName.setName("example.example");
    domainName.setLaunch(null);

    JsonNode json = mapper.valueToTree(domainName);

    assertTrue(json.has("name"));
    assertFalse(json.has("launch"));
    assertFalse(json.has("phase"));
}

@Test
void deserializationFailsWhenRequiredExtensionPropertyIsMissing() {
    String json = "{\"applicationId\":\"abc123\"}";

    assertThrows(MismatchedInputException.class,
        () -> mapper.readValue(json, LaunchExtension.class));
}
```

...

Acknowledgements

TODO

Authors' Addresses

Maarten Wullink

SIDN Labs

Email: maarten.wullink@sidn.nl

URI: <https://sidn.nl/>

Pawel Kowalik

DENIC

Email: pawel.kowalik@denic.de

URI: <https://denic.de/>